AlphaSchema: Exploring the Space of Trading Semantics for LLM-Based Alpha Mining

Anonymous Authors

Abstract

Automated alpha mining has recently shifted from operator- and formula-based search toward large language model (LLM) agents. However, many LLM-based systems lack an explicit exploration space and a principled search policy, relying instead on LLM agents to infer new directions through generation and self-reflection. As a result, exploration remains largely implicit and difficult to control or optimize systematically. We introduce AlphaSchema, which constructs and explores a structured space of trading semantics for alpha mining. Each point in this space is a schema plan composed of Event, Context, Qualities, Direction, and Output, specifying the semantics of a candidate factor before implementation. AlphaSchema decouples exploration from realization: an LLM translates selected schema plans into executable factors, while their rewards are accumulated to learn a surrogate over the semantic space. An iterative selection policy uses this model to balance reward-seeking exploitation with uncertainty, novelty, and local refinement. Experiments on the Chinese stock market show that AlphaSchema discovers factor pools with strong portfolio performance. Further analyses of the semantic evolution of schema plans show that AlphaSchema dynamically explores new regions of the trading-semantic space while gradually concentrating on high-reward regions. Within our framework, the quality of discovered factors is largely decoupled from LLM capability, enabling scalable alpha mining with more economical models.

1 Introduction

Alpha mining aims to discover trading signals that are predictive, executable, interpretable, and stable out of sample. Traditional asset-pricing research derives factors from economic hypotheses and empirical anomalies (Sharpe 1964; Fama and French 1992; 1993),but the vast and evolving space of potential trading mechanisms makes manual discovery inherently limited.[citation needed]. An automated factor-mining system should therefore not only generate candidate signals, but also efficiently explore meaningful trading hypotheses while preserving economic interpretation, implementation validity, and reproducible evaluation.

Existing automated systems have expanded alpha discovery through formulaic and evolutionary search. Formulaic and evolutionary methods represent factors as expressions, syntax trees, or operator programs (Kakushadze 2016; Koza 1992; Su, Lin, and Zhang 2022; Zhang et al. 2020; Cui et al. 2021; Yu et al. 2023; Shi et al. 2025), enabling large-scale exploration of implementation spaces. However, the search process is primarily driven by mathematical composition rather than explicit economic semantics, making it difficult to incorporate higher-level trading hypotheses into the discovery process.

More recent LLM-based agents introduce planning, code generation, memory system, trajectory evolution, and guarded execution into factor discovery (Li et al. 2024; Tang et al. 2025; Shi, Duan, and Li 2025; Lin et al. 2026; Han et al. 2026; Shi et al. 2026; Liu et al. 2025; Chen et al. 2025). However, existing LLM-based systems often delegate both factor realization and exploration decisions to the LLM agent itself. While agentic workflows introduce semantic reasoning into factor discovery, they typically lack an explicitly defined search space, principled exploration algorithms, and structural constraints on factor construction. As a result, the discovery process becomes intertwined with model-specific reasoning traces and workflow designs, making exploration difficult to learn, diagnose, and reproduce. Moreover, the effectiveness of such systems is closely tied to the capability of the underlying LLMs, requiring expensive frontier models and limiting the scalability of large-scale automated mining.

We propose AlphaSchema, a semantic schema-guided evolution framework that separates what to search from how to implement it. Instead of treating code or formulas as the primary search object, AlphaSchema represents each candidate as a structured trading-semantic plan

$$p = (e, c, Q, d, o), \tag{1}$$

where e denotes a market event, c its conditioning context, Q a set of quality constraints, d the directional hypothesis, and o the output form. The search process operates over this schema space, while an LLM trans-

lates selected plans into executable factor implementations for evaluation.

AlphaSchema maintains a reward buffer of evaluated plans and trains a LightGBM ensemble surrogate over schema features (Ke et al. 2017). For unseen plans, the surrogate estimates both predicted reward and ensemble-disagreement uncertainty. A quota selector allocates the limited implementation and backtesting budget across high-reward candidates, uncertain candidates, structurally novel candidates, and mutation-based refinements around historically strong plans. By associating evaluation feedback with schema plans rather than generated code or operators, AlphaSchema learns reusable priors over which trading semantics are worth exploring.

Our contributions are:

- We introduce structured trading-semantic plans as a new search abstraction for alpha mining, moving exploration from implementation artifacts to explicit semantic hypotheses.
- We formulate semantic exploration as a learnable optimization problem, using accumulated plan evaluations and surrogate-based search to balance exploitation, uncertainty, novelty, and refinement under limited budgets.
- We demonstrate that the trading-semantic space is informative and navigable, and that factor quality is largely independent of the choice of LLM backend, enabling scalable alpha mining with economical models.

2 Preliminary

Alpha Mining

An alpha factor is a rule that transforms historical market information into a cross-sectional signal for asset ranking or prediction. Given N tradable assets observed over T trading dates, let $X_t \in \mathbb{R}^{N\times D}$ denote the market state at date t, where rows correspond to assets and columns correspond to observable market features. A factor f maps a historical window of market states into a signal vector:

$$v_t = f(X_{t-\tau+1:t}), \quad v_t \in \mathbb{R}^N.$$

Beyond its executable implementation, a factor typically reflects an underlying market mechanism that provides an interpretable rationale for its predictive relationship with future returns. Therefore, alpha mining involves not only finding predictive transformations, but also discovering meaningful mechanisms and realizing them as executable factors.

Formally, we distinguish a semantic factor plan from its executable realization. Let $p \in \mathcal{P}$ denote a candidate trading-semantic plan, and let

$$R : \mathcal{P} \to \Delta(\mathcal{F})$$

denote a realization process that maps a semantic plan to a distribution over executable factors. A realized factor $f \sim R(p)$ is evaluated by a factor-quality objective $\mathcal{J}(f)$. The objective of semantic factor discovery can therefore be written as

$$p^* = \arg\max_{p\in\mathcal{P}} \mathbb{E}_{f\sim R(p)}[\mathcal{J}(f)].$$

A plan does not uniquely determine an executable factor: different realizations may adopt different operators, parameters, and functional forms. Thus, the objective concerns the reward distribution induced by a plan rather than a deterministic mapping.

Structured Semantic Plans

To search over trading semantics, AlphaSchema represents each candidate factor as a structured semantic plan. Rather than assuming a unique ontology of market mechanisms, we define a practical and expressive representation that captures recurring dimensions of factor construction. A semantic plan describes the intended trading mechanism before implementation and can subsequently be translated into an executable factor by an LLM.

Formally, a semantic plan is represented as

$$p = (e, c, Q, d, o) \in \mathcal{P},$$

where the five dimensions jointly characterize different aspects of a candidate trading mechanism:

- Event: the market phenomenon or signal-generating event being captured, such as breakout, volume expansion, or volatility compression;
- Context: the market state or reference condition under which the event is interpreted, such as a recent extreme, VWAP region, volatility regime, or cross-sectional quantile;
- Qualities: additional properties or validation criteria that refine the mechanism, such as volume confirmation, multi-horizon consistency, or outlier filtering;
- Direction: the expected relationship between the mechanism and future returns, such as continuation, reversal, or range oscillation;
- Output: the form in which the mechanism is expressed as a tradable signal, such as a continuous score, event decay, or cross-sectional rank.

Together, these dimensions specify what market phenomenon is considered, under which conditions it is interpreted, what evidence supports it, how it is expected to affect future returns, and how the resulting signal is expressed. Unlike formula trees or sequential decision processes, a semantic plan does not impose a predefined ordering or compositional structure among these dimensions. They jointly define a point in the trading-semantic space, allowing exploration without introducing artificial constraints on the underlying market mechanisms.

3 Methodology

Given the trading-semantic space defined in Section 2, AlphaSchema performs iterative search under a fixed generation and backtesting budget. In each round, candidate plans are proposed and selected using predicted reward, uncertainty, novelty, and local mutation. Selected plans are realized as executable factors, validated, and backtested. Their rewards are added to a buffer to update the semantic surrogate for the next round.

The search loop has three components: (1) semantic plan proposal and selection, which combines global sampling, local mutation, and quota-based allocation; (2) plan realization and evaluation, which generates executable factors and applies execution, leakage, and backtesting checks; and (3) semantic reward learning, which learns from accumulated plan–reward observations to score unevaluated plans. A separate post-search procedure removes weak or redundant factors for downstream evaluation. Figure 1 summarizes the closed-loop workflow.

Semantic Plan Space Construction

We instantiate a trading-semantic space using configurable vocabularies for events, contexts, qualities, directions, and outputs. Let these vocabularies be denoted by $\mathcal{V}_E$, $\mathcal{V}_C$, $\mathcal{V}_Q$, $\mathcal{V}_D$, and $\mathcal{V}_O$, respectively. The resulting plan space is

$$\mathcal{P} = \mathcal{V}_E \times \mathcal{V}_C \times \mathcal{V}_Q \times \mathcal{V}_D \times \mathcal{V}_O.$$

The vocabularies can be derived from domain expertise, academic and practitioner literature, existing factor libraries, and findings from previous discovery runs. They need not cover all trading semantics and can instead be configured for a target domain, such as equity fundamentals or futures term structure.

A plan describes only trading logic, without prescribing operators, library functions, or numerical parameters. We also impose no hand-crafted compatibility rules across dimensions. Unusual or potentially inconsistent combinations are evaluated through code realization and empirical feedback, avoiding complex prior constraints while retaining unconventional combinations that may still produce useful factors.

Plan Realization and Execution Validation

Given a selected plan p, the code agent receives its semantic specification, data contract, and implementation constraints, and translates it into executable factor functions. The plan specifies only the intended trading logic; the agent independently chooses the operators, functional form, and numerical parameters. It implements each plan at a fast and a slow time scale, yielding realizations $f_{p,\text{fast}}$ and $f_{p,\text{slow}}$.

Before backtesting, each realization passes through execution guards that verify contract compliance and computational validity, test numerical stability, and screen for potential look-ahead leakage. Invalid implementations are excluded before reward computation, preventing erroneous results from contaminating semantic feedback.

Semantic Reward Modeling

For each valid realization $f_{p,s}$ of plan p, where $s \in \{\text{fast}, \text{slow}\}$, we define

$$r_{p,s} = \alpha\,\text{RankIC}(f_{p,s}) + \beta\,\text{RankICIR}(f_{p,s}) - \lambda\,\Delta_{\text{lag}}(f_{p,s}), \tag{2}$$

where

$$\Delta_{\text{lag}}(f) = \max\{0,\, \text{RankIC}(f) - \text{RankIC}(L_1 f)\},$$

and $L_1 f$ denotes the signal delayed by one period. The coefficients $\alpha, \beta, \lambda > 0$ control predictive strength, temporal stability, and lag sensitivity, respectively. The plan reward is

$$r(p) = \max_{s \in \{\text{fast},\text{slow}\}} r_{p,s}.$$

Because realization is stochastic, $r(p)$ is a noisy observation of plan-level quality rather than a deterministic target. Within each run, we keep the LLM backend, data contract, and validation pipeline fixed. The resulting realization policy is therefore consistent across plans, allowing the surrogate to learn shared reward tendencies from the growing buffer.

After each round, new plan–reward pairs are added to the reward buffer

$$\mathcal{D}_t = \mathcal{D}_{t-1} \cup \{(p_i, r(p_i))\}_{i \in \mathcal{B}_t}.$$

A LightGBM ensemble is retrained on the expanded buffer using structured plan features $\phi(p)$, and produces a predicted reward mean $\mu_t(p)$ and ensemble uncertainty $\sigma_t(p)$ for unevaluated plans.

Adaptive Quota-Based Plan Selection

Let K denote the number of plans evaluated in each round. We divide this budget among global exploration, reward-based selection, and local mutation. If n_t plans have been evaluated before round t, the exploration ratio is

$$\rho_t = \rho_{\min} + (\rho_{\max} - \rho_{\min}) \exp\!\left(-\frac{n_t}{\tau}\right),$$

where

$$0 \le \rho_{\min} \le \rho_{\max} \le 1, \qquad \tau > 0.$$

The round-wise quotas are then

$$K_t^{\text{explore}} = \lfloor K\rho_t \rceil,$$
$$K_t^{\text{reward}} = \left\lfloor \frac{K - K_t^{\text{explore}}}{2} \right\rfloor,$$
$$K_t^{\text{mut}} = K - K_t^{\text{explore}} - K_t^{\text{reward}}.$$

The exploration channel selects structurally novel unseen plans. The reward channel selects unseen plans

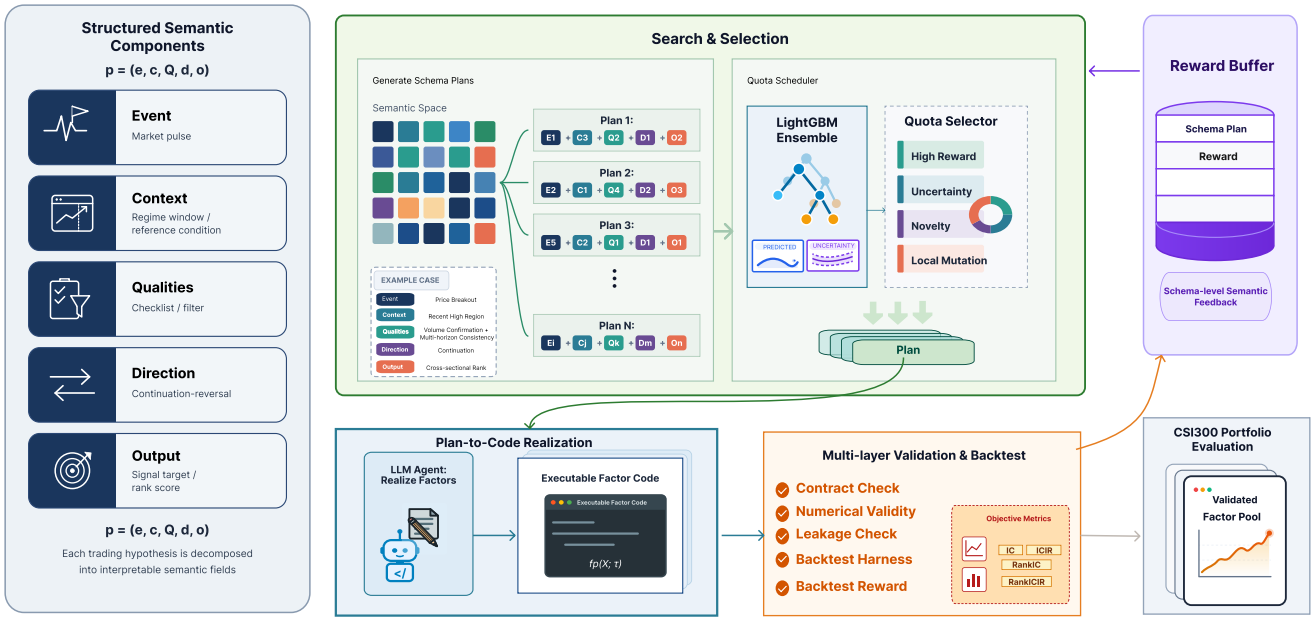

Figure 1: Overview of AlphaSchema. The framework separates semantic search from executable realization: candidate trading hypotheses are represented as structured plans over event, context, qualities, direction, and output, selected by a reward-guided semantic scheduler, realized into executable factor code by an LLM agent, and evaluated through guarded backtesting. The resulting schema–reward pairs are stored in a reward buffer and used to update the schema-level scorer for subsequent selection rounds.

with high surrogate-predicted rewards. The mutation channel selects high-reward plans from the evaluated set and modifies one schema component to generate local variants.

As the reward buffer grows, the schedule shifts from broad exploration toward reward-guided selection and mutation. Mutation is particularly important because alpha mining aims to discover individual high-quality factors. It directly refines successful plans and serves as an important source of strong factors.

4 Experiments

We first compare the discovered factor pools with representative predictive models, factor libraries, and agentic mining systems. We then examine the contribution of individual schema components, analyze realization variability and code-agent robustness, and test whether schema features predict rewards before implementation.

Experimental Setup

Dataset and Metrics. Following recent Qlib-based alpha-mining studies, we use the CSI300 universe as the primary benchmark. The time span is split into training (Jan 1, 2016 – Dec 31, 2020), validation (Jan 1, 2021 – Dec 31, 2022), and testing (Jan 1, 2023 – Dec 31, 2025). The prediction target is the 5-day forward close-to-close return computed from daily OHLCV-style market data. We report factor predictive metrics (IC, ICIR, RankIC, RankICIR) and strategy metrics based on benchmark-relative excess returns (AER, IR, MDD). Data-processing, trading-cost, and metric definitions are provided in Appendix A.

Baselines. We compare against four families of baselines. Machine-learning predictors include MLP and XGBoost (Chen and Guestrin 2016). Deep sequence models include Transformer (Vaswani et al. 2017), GRU (Cho et al. 2014), and LSTM (Hochreiter and Schmidhuber 1997). Open-source factor libraries include Alpha158 and Alpha360 (Yang et al. 2020). Agentic mining systems include RD-Agent (Li et al. 2025), QuantaAlpha (Han et al. 2026), and AlphaPROBE (Guo et al. 2026). For reproduced agentic mining runs and AlphaSchema, the LLM backend is fixed to the GPT-5.4-mini API under matched call and candidate-evaluation budgets. Predictor baselines are trained directly under the same train/validation/test split. Since the compared baselines do not support controlled fundamental-schema mining, we report both an OHLCV-only version of AlphaSchema for the fair comparison and a +Fundamental version to show the additional schema modality supported by our framework.

Main Results

Pool-Level Evaluation The principal comparison for AlphaSchema is not at the level of a single factor, but at the level of the exported factor pool. Table 1 reports factor predictive power and strategy performance

Table 1: Main-result table on CSI300. "Fund. Schema" indicates whether the mining process is allowed to use fundamental semantic schema fields. Higher is better for all metrics except MDD. Best results are bolded and second-best results are underlined; the pending AlphaPROBE reproduced run is excluded from ranking.

Category	Method	Fund. Schema	Factor Predictive Power				Strategy Performance		
			IC	ICIR	Rank IC	Rank ICIR	IR	AER (%)	MDD (%)↓
Machine-Learning Predictors									
Machine Learning	MLP	✗	0.0236	0.1408	0.0389	0.2471	0.5901	5.16	11.47
	XGBoost	✗	0.0284	0.2024	0.0373	0.2699	0.3556	2.55	<u>10.30</u>
Deep Sequence Models									
Deep Learning	Transformer	✗	0.0289	0.1561	0.0507	0.2775	0.6929	6.14	12.03
	GRU	✗	0.0310	0.1782	<u>0.0565</u>	<u>0.3242</u>	0.4609	3.73	13.21
	LSTM	✗	<u>0.0380</u>	0.2269	0.0587	0.3521	0.6317	5.14	11.51
Open-Source Factor Libraries									
Factor Library	Alpha158	✗	0.0347	0.2081	0.0504	0.3009	0.5474	4.36	15.87
	Alpha360	✗	0.0231	0.1710	0.0292	0.2100	0.4024	2.99	8.86
Agentic Mining Systems									
Agentic Mining	RD-Agent	✗	0.0242	0.1494	0.0504	0.3094	<u>0.9861</u>	6.81	15.57
	QuantaAlpha	✗	0.0208	0.1619	0.0380	0.2934	0.6726	5.57	14.18
Ours	AlphaSchema (OHLCV)	✗	0.0382	0.2374	0.0498	0.2912	0.7624	<u>8.53</u>	18.63
	AlphaSchema (+Fund.)	✓	<u>0.0380</u>	<u>0.2365</u>	0.0487	0.2857	1.0877	11.94	15.43

under the held-out test period. Because existing baselines do not support fundamental-schema factor mining, the OHLCV-only AlphaSchema row is the fairness-aligned comparison against prior methods. It achieves the strongest IC (0.0382) and ICIR (0.2374), while using the same broad market-data modality as the baselines. The +Fundamental row then activates an additional schema family supported by our framework and obtains the strongest portfolio-level IR (1.0877) and AER (11.94%). This two-row reporting separates a fair like-for-like comparison from a capability demonstration: AlphaSchema can search over fundamental accounting semantics when such data are available, whereas the compared methods cannot directly express or protect that schema during mining. LSTM attains the highest Rank IC and Rank ICIR among the listed baselines, but its strategy return remains substantially lower. This pattern suggests that the discovered factor pool is especially effective after portfolio construction: its advantage is not only cross-sectional correlation strength, but also the conversion of predictive signal into benchmark-relative excess return.

Figure 2 provides a complementary library-level view of the same held-out period. It compares the cumulative excess return of the exported AlphaSchema top-150 pool with representative open-source factor libraries under the same CSRankNorm–LightGBM–Top50/Drop5 protocol. The AlphaSchema curve shows a more persistent benchmark-relative wealth spread, indicating that the selected factor pool converts predictive signal into stronger portfolio-level excess return.

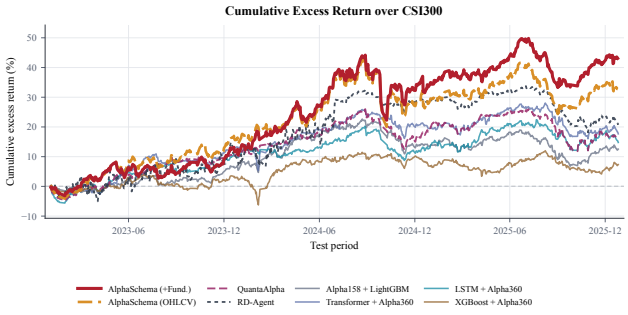

Figure 2: Held-out cumulative excess return against representative open-source factor libraries on CSI300 over the 2023–2025 test period. All curves are evaluated under the same CSRankNorm, LightGBM combiner, and Top50/Drop5 backtest protocol.

Ablation Study

We first test whether each semantic schema field is necessary. Unless otherwise noted, all variants share the same search budget, LLM backbone, downstream LightGBM combiner, and backtest protocol.

Component Ablation of the Schema Grammar The second group targets the semantic grammar itself. We fix a set of 100 high-reward full schema plans and construct five blind leave-one-out variants for each plan, removing exactly one field from {Event, Context, Qualities, Direction, Output}. The code agent observes only the remaining fields and is not told which field is missing. This design isolates the contribution of individual schema fields from differences in search trajectory: all

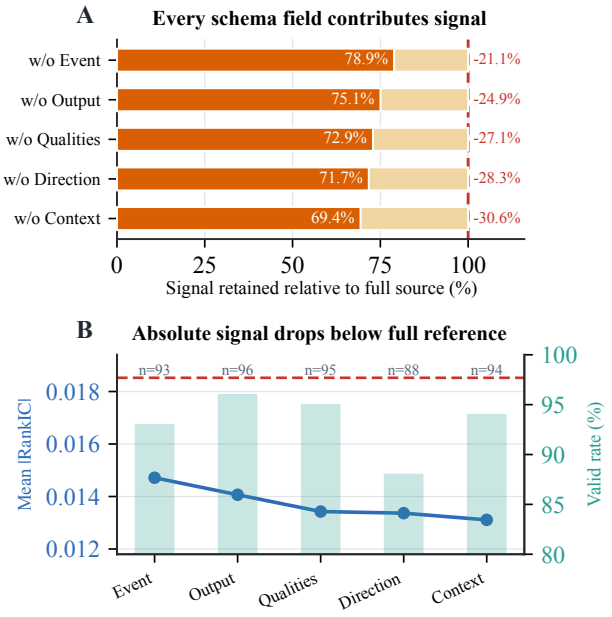

Figure 3: Component-wise schema ablation.

variants originate from the same full-plan set, while degradation is measured after guarded implementation and factor evaluation. Since a single informative factor can be useful with either sign, we use mean absolute Rank IC as the primary diagnostic metric.

Figure 3 summarizes the leave-one-field-out results. Panel A reports retained signal strength relative to the full source reference, and Panel B separates mean |RankIC| from implementation validity. All five removals reduce signal strength, retaining only 69.4%–78.9% of the full-plan reference. The largest degradation occurs when Context is removed, suggesting that regime and reference conditioning are central to converting an economic event into a useful cross-sectional signal. The schema benefit is therefore distributed across fields rather than carried by a single descriptor.

Backend Robustness Analysis

We test whether schema-guided realization depends on a single implementation agent. Holding 100 schema plans fixed, six code-agent backends implement the same plans, with at most one retry for failed cases when available. Final success rates range from 82% to 99%, while mean |RankIC| over successful implementations remains in a comparable range (0.0116–0.0168). Figure 4 separates these two effects: Panel A reports pass@1 success and repair recovery, and Panel B reports post-success signal quality. The result suggests that backend choice mainly affects execution reliability, while the semantic plan and the resulting factor function transfer across different code agents. Appendix B provides the full table.

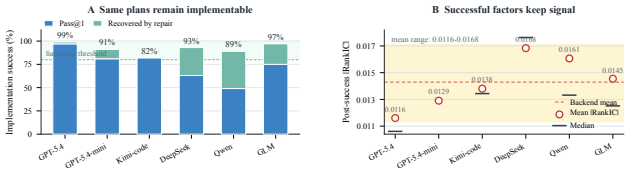

Figure 4: Repair-aware robustness to code-agent backends on the same 100 schema plans. Panel A separates first-attempt implementation success from repair recovery; Panel B shows that successful implementations retain comparable signal quality across backends.

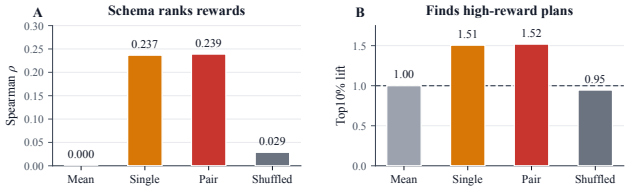

Figure 5: Schema-space reward predictability.

Schema-Space Predictability Analysis

We further ask whether schema rewards are predictable before code realization. We collect 12,126 historical plan–reward pairs covering 11,295 unique schema keys and train lightweight predictors using only discrete schema features. Single schema uses component-level indicators and category/scope counts, while Schema + pair additionally includes pairwise component interactions. No schema text embeddings, generated code, factor values, or backtest trajectories are used as inputs.

Figure 5 shows that schema-level features are predictive under this restricted setting. Panel A compares reward-ranking quality, while Panel B reports top-decile enrichment for the mean baseline, the single-schema LightGBM scorer, the pair-interaction LightGBM scorer, and the shuffled-reward control. Single-schema LightGBM reaches Spearman 0.237 and top-decile lift 1.506, while adding pair interactions improves the lift to 1.519. The mean baseline has no ranking ability, and the shuffled-reward control remains near random, indicating that the signal is tied to the true schema–reward association rather than model capacity alone.

5 Related Work

Automated alpha mining has evolved from human-crafted factors and symbolic heuristics to large-scale algorithmic search. Early work formalized interpretable return predictors through factor models and anomaly studies (Fama and French 1992; Harvey, Liu, and Zhu 2016; Hou, Xue, and Zhang 2020). Later systems automated formula discovery through genetic programming, reinforcement learning, and generative search (Su, Lin, and Zhang 2022; Zhang et al. 2020; Cui et al. 2021;

Yu et al. 2023; Shi et al. 2025). These approaches improved coverage of the factor space, but their search objects remained formulas or program structures, making it difficult to reason explicitly about the underlying trading semantics before implementation.

Recent LLM-based systems substantially extend this line of work. FAMA improves in-context factor generation through cross-sample selection and chain-of-experience reuse (Li et al. 2024). AlphaAgent introduces originality, hypothesis-factor consistency, and complexity regularization to mitigate alpha decay (Tang et al. 2025). MCTS-Alpha reframes alpha discovery as tree search over formulas, while FactorEngine shifts the search object from formulas to executable programs (Shi, Duan, and Li 2025; Lin et al. 2026). QuantaAlpha evolves full mining trajectories through mutation and crossover, Hubble emphasizes guarded execution and family-aware selection, CogAlpha pushes the search deeper into code-level evolution, and AlphaSAGE explicitly targets diverse high-reward alpha libraries through structure-aware generative search (Han et al. 2026; Shi et al. 2026; Liu et al. 2025; Chen et al. 2025). Collectively, these methods show that LLMs are useful not merely as text generators but as components in structured discovery systems. However, their selectors still mostly operate on formulas, programs, or trajectories after implementation has already been attempted.

In parallel, agentic finance and self-evolving systems have highlighted the importance of memory, structured feedback, and execution-grounded evaluation. R&D-Agent-Quant decouples research and development stages for data-centric factor-model co-optimization (Li et al. 2025). TradingAgents, FinMem, and FinCon demonstrate the value of role specialization, layered memory, and long-horizon consistency in financial reasoning workflows (Xiao et al. 2024; Yu et al. 2025; 2024). Beyond finance, AlphaEvolve and controlled self-evolution systems show that LLMs can iteratively improve code under evaluator feedback (Novikov et al. 2025; Hu et al. 2026). Our framework is motivated by these trends, but differs in one central aspect: the primary search object in AlphaSchema is an explicit semantic plan over event, context, qualities, direction, and output. This allows the system to learn which economic mechanisms are promising before paying full implementation cost, while still retaining executable validation at the code level.

6 Conclusion

We presented AlphaSchema, a semantic schema-guided evolution framework for LLM-based alpha mining. By factorizing candidate hypotheses into event, context, qualities, direction, and output components, the framework searches over interpretable trading semantics before code generation. A constrained implementation agent, execution guards, executable rewards, and an online plan-level scorer together form a closed loop for allocating search budget across exploitation, uncertainty, and novelty. Experiments on CSI300 show that the resulting factor pool improves portfolio-level excess return, that each schema field contributes measurable signal, and that schema-level rewards are predictable before code realization. Backend-robustness results further indicate that the semantic plans transfer across multiple code-agent implementations once execution succeeds.

References

Chen, T., and Guestrin, C. 2016. Xgboost: A scalable tree boosting system. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785–794. ACM.

Chen, B.; Ding, H.; Shen, N.; Huang, J.; Guo, T.; Liu, L.; and Zhang, M. 2025. Alphasage: Structure-aware alpha mining via gflownets for robust exploration. arXiv preprint arXiv:2509.25055.

Cho, K.; van Merriënboer, B.; Gulcehre, C.; Bahdanau, D.; Bougares, F.; Schwenk, H.; and Bengio, Y. 2014. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, 1724–1734.

Cui, C.; Wang, W.; Zhang, M.; Chen, G.; Luo, Z.; and Ooi, B. C. 2021. Alphaevolve: A learning framework to discover novel alphas in quantitative investment. In Proceedings of the 2021 International Conference on Management of Data, 2208–2216. ACM.

Fama, E. F., and French, K. R. 1992. The cross-section of expected stock returns. The Journal of Finance 47(2):427–465.

Fama, E. F., and French, K. R. 1993. Common risk factors in the returns on stocks and bonds. Journal of Financial Economics 33(1):3–56.

Guo, T.; Shen, H.; Luo, J.; Chen, B.; Ding, H.; Huang, J.; Liu, L.; Ma, Y.; and Zhang, M. 2026. Alphaprobe: Alpha mining via principled retrieval and on-graph biased evolution. arXiv preprint arXiv:2602.11917.

Han, J.; Zhang, S.; Li, W.; Dong, Y.; Hu, T.; Zhu, Y.; Yu, X.; Guo, X.; Liu, Z.; Wang, K.; Liu, J.; Jiang, T.; An, R.; Hu, S.; Yang, Z.; Che, R.; and Wang, H. 2026. Quantaalpha: An evolutionary framework for llm-driven alpha mining. arXiv preprint arXiv:2602.07085.

Harvey, C. R.; Liu, Y.; and Zhu, H. 2016. … and the cross-section of expected returns. The Review of Financial Studies 29(1):5–68.

Hochreiter, S., and Schmidhuber, J. 1997. Long short-term memory. Neural Computation 9(8):1735–1780.

Hou, K.; Xue, C.; and Zhang, L. 2020. Replicating anomalies. The Review of Financial Studies 33(5):2019–2133.

Hu, T.; Chen, R.; Zhang, S.; Yin, J.; Feng, M. X.; Liu, J.; Zhang, S.; Jiang, W.; Fang, Y.; Hu, S.; Wang, H.; and Xu, Y. 2026. Controlled self-evolution

for algorithmic code optimization. arXiv preprint arXiv:2601.07348.

Kakushadze, Z. 2016. 101 formulaic alphas. Wilmott 2016(84):72–81.

Ke, G.; Meng, Q.; Finley, T.; Wang, T.; Chen, W.; Ma, W.; Ye, Q.; and Liu, T.-Y. 2017. Lightgbm: A highly efficient gradient boosting decision tree. In Advances in Neural Information Processing Systems 30, 3146–3154.

Koza, J. R. 1992. Genetic Programming: On the Programming of Computers by Means of Natural Selection. MIT Press.

Li, Z.; Song, R.; Sun, C.; Xu, W.; Yu, Z.; and Wen, J.-R. 2024. Can large language models mine interpretable financial factors more effectively? a neural-symbolic factor mining agent model. In Findings of the Association for Computational Linguistics: ACL 2024, 3891–3902. Association for Computational Linguistics.

Li, Y.; Yang, X.; Yang, X.; Xu, M.; Wang, X.; Liu, W.; and Bian, J. 2025. R&d-agent-quant: A multi-agent framework for data-centric factors and model joint optimization. arXiv preprint arXiv:2505.15155.

Lin, Q.; Feng, R.; Feng, Y.; Huang, Z.; Chen, Y.; Yang, Z.; Zhou, L.; Fei, B.; Liu, J.; and Li, Y. 2026. Factorengine: A program-level knowledge-infused factor mining framework for quantitative investment. arXiv preprint arXiv:2603.16365.

Liu, F.; Huang, Y.; Luo, S.; Wang, Y.; Yang, Y.; Li, X.; Hu, Z.; Feng, J.; and Liu, Q. 2025. Cognitive alpha mining via llm-driven code-based evolution. arXiv preprint arXiv:2511.18850.

Novikov, A.; Vu, N.; Eisenberger, M.; Dupont, E.; Huang, P.-S.; Wagner, A. Z.; Shirobokov, S.; Kozlovskii, B.; Ruiz, F. J. R.; Mehrabian, A.; et al. 2025. Alphaevolve: A coding agent for scientific and algorithmic discovery. arXiv preprint arXiv:2506.13131.

Sharpe, W. F. 1964. Capital asset prices: A theory of market equilibrium under conditions of risk. The Journal of Finance 19(3):425–442.

Shi, H.; Song, W.; Zhang, X.; Shi, J.; Luo, C.; Ao, X.; Arian, H.; and Seco, L. A. 2025. Alphaforge: A framework to mine and dynamically combine formulaic alpha factors. In Proceedings of the AAAI Conference on Artificial Intelligence, volume 39, 12524–12532.

Shi, R.; Yan, S.; Cai, Y.; and Lv, C. 2026. Hubble: An llm-driven agentic framework for safe, diverse, and reproducible alpha factor discovery. arXiv preprint arXiv:2604.09601.

Shi, Y.; Duan, Y.; and Li, J. 2025. Navigating the alpha jungle: An llm-powered mcts framework for formulaic factor mining. arXiv preprint arXiv:2505.11122.

Su, Z.; Lin, J.; and Zhang, C. 2022. Genetic algorithm based quantitative factors construction. In 2022 IEEE 20th International Conference on Industrial Informatics (INDIN), 650–655. IEEE.

Tang, Z.; Chen, Z.; Yang, J.; Mai, J.; Zheng, Y.; Wang, K.; Chen, J.; and Lin, L. 2025. Alphaagent: Llm-driven alpha mining with regularized exploration to counteract alpha decay. In Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2813–2822. ACM.

Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; Kaiser, Ł.; and Polosukhin, I. 2017. Attention is all you need. In Advances in Neural Information Processing Systems.

Xiao, Y.; Sun, E.; Luo, D.; and Wang, W. 2024. Tradingagents: Multi-agents llm financial trading framework. arXiv preprint arXiv:2412.20138.

Yang, X.; Liu, W.; Zhou, D.; Bian, J.; and Liu, T.-Y. 2020. Qlib: An ai-oriented quantitative investment platform. arXiv preprint arXiv:2009.11189.

Yu, S.; Xue, H.; Ao, X.; Pan, F.; He, J.; Tu, D.; and He, Q. 2023. Generating synergistic formulaic alpha collections via reinforcement learning. In Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 5476–5486. ACM.

Yu, Y.; Yao, Z.; Li, H.; Deng, Z.; Jiang, Y.; Cao, Y.; Chen, Z.; Suchow, J. W.; Cui, Z.; Liu, R.; et al. 2024. Fincon: A synthesized llm multi-agent system with conceptual verbal reinforcement for enhanced financial decision making. Advances in Neural Information Processing Systems 37:137010–137045.

Yu, Y.; Li, H.; Chen, Z.; Jiang, Y.; Li, Y.; Suchow, J. W.; Zhang, D.; and Khashanah, K. 2025. Finmem: A performance-enhanced llm trading agent with layered memory and character design. IEEE Transactions on Big Data.

Zhang, T.; Li, Y.; Jin, Y.; and Li, J. 2020. Autoalpha: An efficient hierarchical evolutionary algorithm for mining alpha factors in quantitative investment. arXiv preprint arXiv:2002.08245.

A Evaluation Details

Experimental Protocol

We use CSI300 as the stock universe and SH000300 as the benchmark index. The time span is split into training (Jan 1, 2016 – Dec 31, 2020), validation (Jan 1, 2021 – Dec 31, 2022), and testing (Jan 1, 2023 – Dec 31, 2025). The main label is the 5-day forward return:

$$y_{i,t} = \frac{\text{Ref}(\text{close}, -6)}{\text{Ref}(\text{close}, -1)} - 1. \tag{3}$$

Factor selection, correlation filtering, and model selection are performed only on the training and validation windows. The 2023–2025 period is reserved for held-out reporting.

Downstream Combiner for Factor Pools

For methods that produce an explicit factor pool, the selected factors are combined by a LightGBM ranker. Given an exported factor pool $\mathcal{F} = \{f_1, \ldots, f_K\}$, we construct

$$z_{i,t} = [f_1(i,t), \ldots, f_K(i,t)] \tag{4}$$

for stock i on day t. Missing values and infinite values are processed before training, invalid labels are removed, and both features and labels are normalized by cross-sectional rank normalization. LightGBM is trained on the training window, while early stopping and hyperparameter selection use the validation window before test evaluation. The factor-pool experiments use 500 boosting rounds, early stopping after 50 rounds without validation improvement, and a fixed hyperparameter configuration across the corresponding factor-pool runs.

Predictive Metrics

We follow the standard factor-evaluation convention used in Qlib-style alpha-mining papers and report both linear-correlation and rank-correlation metrics. For each test day t, let $\hat{y}_{i,t}$ be the model score, $y_{i,t}$ the realized label, and $\mathcal{U}_t$ the tradable cross-section after suspension, missing-label, and validity filters.

Information Coefficient (IC). IC measures whether predicted scores are linearly aligned with realized cross-sectional returns:

$$\begin{aligned} \mathrm{IC}_t &= \mathrm{corr}_{i\in\mathcal{U}_t}(\hat{y}_{i,t}, y_{i,t}), \\ \mathrm{IC} &= \frac{1}{|\mathcal{T}_{\mathrm{test}}|}\sum_{t\in\mathcal{T}_{\mathrm{test}}} \mathrm{IC}_t. \end{aligned} \tag{5}$$

IC Information Ratio (ICIR).

$$\mathrm{ICIR} = \frac{\mathrm{mean}_t(\mathrm{IC}_t)}{\mathrm{std}_t(\mathrm{IC}_t)}. \tag{6}$$

Rank Information Coefficient (Rank IC).

$$\begin{aligned} \mathrm{RankIC}_t &= \mathrm{corr}_{i\in\mathcal{U}_t}\big(\mathrm{rank}(\hat{y}_{i,t}), \mathrm{rank}(y_{i,t})\big), \\ \mathrm{RankIC} &= \frac{1}{|\mathcal{T}_{\mathrm{test}}|}\sum_{t\in\mathcal{T}_{\mathrm{test}}} \mathrm{RankIC}_t. \end{aligned} \tag{7}$$

Rank IC Information Ratio (Rank ICIR).

$$\mathrm{RankICIR} = \frac{\mathrm{mean}_t(\mathrm{RankIC}_t)}{\mathrm{std}_t(\mathrm{RankIC}_t)}. \tag{8}$$

Predictive metrics are computed on the held-out test/backtest window and do not include transaction costs. Days with too few tradable assets or degenerate prediction/label vectors are excluded.

Trading Strategy

Strategy-level evaluation uses Qlib's TopkDropoutStrategy with topk=50 and n_drop=5. On each rebalance date, stocks are ranked by $\hat{y}_{i,t}$. The target portfolio holds the top 50 names with equal weights. To reduce turnover, existing holdings that remain sufficiently highly ranked are retained; at most five lowest-ranked current holdings are replaced by the highest-ranked non-held names. Rebalancing is daily. This Top50/Drop5 rule follows recent Qlib-based alpha-mining protocols and keeps the comparison focused on signal quality rather than portfolio optimization.

Execution Assumptions and Costs

Trades are executed at the open price on the trading day after signal generation (deal_price=open). Transaction costs are modeled as a buying cost of 0.05% and a selling cost of 0.15%, with a minimum fee of 5 currency units per trade and a round-trip friction of approximately 0.20%. Limit-up/limit-down filtering is enabled with limit_threshold=0.095. Untradable names are skipped for new purchases, while existing holdings are handled according to Qlib exchange rules.

Portfolio Metrics

Predictive metrics measure ranking quality but not tradability. We therefore report portfolio-level metrics under the Top50/Drop5 strategy. Let r_t^{port} be the portfolio return, r_t^{bench} the CSI300 benchmark return, and c_t the transaction-cost term. The daily excess return is defined as

$$e_t = r_t^{\mathrm{port}} - r_t^{\mathrm{bench}} - c_t, \tag{9}$$

where the benchmark is SH000300. All portfolio metrics below are computed from $\{e_t\}$ with $N = 252$ trading days per year.

Annualized Excess Return (AER).

$$\mathrm{AER} = \left(\prod_{t=1}^{T}(1+e_t)\right)^{252/T} - 1. \tag{10}$$

AER is therefore the annualized benchmark-relative excess return after transaction costs, rather than the annualized raw portfolio return.

Information Ratio (IR).

$$\mathrm{IR} = \frac{\mathrm{mean}_t(e_t)}{\mathrm{std}_t(e_t)}\sqrt{252}. \tag{11}$$

Because the excess return is defined relative to CSI300 rather than a risk-free rate, IR measures benchmark-relative risk-adjusted performance.

Maximum Drawdown (MDD). MDD is computed on the cumulative excess-return curve induced by $\{e_t\}$:

$$\mathrm{MDD} = \max_{t\le T}\left(1 - \frac{C_t}{\max_{s\le t} C_s}\right), \quad C_t = \prod_{s=1}^{t}(1+e_s). \tag{12}$$

B Additional Ablation Details

Formal Schema Notation

The semantic plan space is written as

$$\mathcal{P} = \mathcal{E} \times \mathcal{C} \times 2^{\mathcal{Q}} \times \mathcal{D} \times \mathcal{O}.$$

Here $\mathcal{P}$ denotes the set of all candidate plans. The five component sets correspond to Event choices $\mathcal{E}$, Context choices $\mathcal{C}$, Quality choices $\mathcal{Q}$, Direction choices $\mathcal{D}$, and Output choices $\mathcal{O}$. We use $2^{\mathcal{Q}}$ for qualities because a plan may include multiple confirmation or filtering conditions, whereas the other fields usually select one primary option.

Table 2: Summary of the exported factor-pool artifact.

Item	Value
Exported factor sources	150
EBS factors	60
Previous-pool factors	90
Most common selected periods	20, 80, 60
Included artifacts	source, manifest, period audit, config, metrics, curves
Excluded artifacts	raw market data, labels, factor-value parquet files

Thus, a plan can be written as $p = (e, c, Q, d, o)$, where $e \in \mathcal{E}$, $c \in \mathcal{C}$, $Q \subseteq \mathcal{Q}$, $d \in \mathcal{D}$, and $o \in \mathcal{O}$. Its canonical key is

$$\text{key}(p) = e \mid c \mid q_1 \mid \cdots \mid q_m \mid d \mid o,$$

which supports deduplication, coverage tracking, mutation, and surrogate scoring.

Factor-Pool Export and Artifact Scope

For auditability, we maintain an export package for the +Fundamental-schema AlphaSchema factor pool reported in Table 1. The current package contains 150 factor source files, consisting of 60 EBS reward factors and 90 factors inherited from a previous top-150 pool after correlation-aware selection; the latter subset includes 68 price-volume factors and 22 fundamental factors. Each exported factor records its selected period, candidate periods, direction adjustment, source group, and source-code mapping. The export also includes selection manifests, period-audit tables, saved Qlib configuration, metric snapshots, and plot data for regenerating reported curves.

This artifact supports source inspection, configuration review, period auditing, and figure regeneration from saved curves. It is not intended as a complete raw-data reproduction package, since exact reruns also depend on the original data preprocessing and factor materialization environment.

Final Factor-Pool Selection

The final modeling pool is selected from the validated factor pool by a rank-and-filter rule. Let $\mathcal{V}$ denote the set of valid candidate factors with available Rank IC scores. Candidates are first sorted in descending order by Rank IC (RIC). We then scan this ordered list greedily and add a candidate f to the selected pool $\mathcal{S}$ only if

$$\max_{g \in \mathcal{S}} |\text{corr}(f, g)| < 0.7, \tag{13}$$

where the correlation is computed from factor values on the pool-selection window. This rule prioritizes individually strong factors while controlling redundancy in the exported pool. The reported export contains 150 factors: 60 newly selected EBS factors and 90 factors inherited from the previous top-150 pool after the same correlation-aware selection procedure.

Table 3: Detailed schema-grammar leave-one-out results. Full is the source full-plan reference used to construct the ablation tasks.

Variant	Missing Field	Valid	**|RankIC|** ↑	Rel. (%)	Reward
AlphaSchema (Full source)	–	100/100	0.0185	100.0	4.174
w/o Event	Event	93/100	0.0147	78.9	2.199
w/o Context	Context	94/100	0.0131	69.4	2.249
w/o Qualities	Qualities	95/100	0.0134	72.9	2.483
w/o Direction	Direction	88/100	0.0134	71.7	2.219
w/o Output	Output	96/100	0.0141	75.1	2.310

Schema Grammar Ablation

The schema ablation is a leave-one-component-out experiment over the five semantic fields E, C, Q, D, O. We sample 100 high-reward full schema plans and construct five blind variants for each plan by removing one field at a time. The implementation agent receives only the visible fields and is not told which component was removed. This yields 500 implementation tasks, of which 466 pass the full validity and evaluation pipeline. The primary metric is mean absolute Rank IC, because this experiment measures whether a generated factor contains ranking information regardless of whether the profitable direction is positive or negative.

The full reference is not reimplemented under the blind ablation prompt; it is read from the source full-plan runs used to build the 100-plan task set. We therefore use this experiment as a component-level diagnostic rather than as a final portfolio-level comparison. A complete paired subset in which all five ablated variants are valid contains 71 source plans and yields the same qualitative ordering: every leave-one-out variant remains below the full source reference.

Figure 3 visualizes the same comparison in the main paper. Every field removal reduces retained signal strength, while most variants still maintain high implementation validity. The degradation is therefore not only an execution failure effect, but also reflects weaker schema semantics.

Code-Agent Backend Robustness

This experiment isolates the effect of the implementation backend. We fix the same 100 schema plans and ask six code agents to realize them. Pass@1 records first-attempt success; failed cases receive at most one retry/repair when available. A run is counted as successful only when the generated factor passes evaluation and receives a valid reward. Signal quality is measured by mean |RankIC| over final successful factors.

The corresponding visualization is reported in Figure 4 in the main paper. The results separate execution reliability from factor quality. Retry mainly improves completion rates for lower Pass@1 backends, while successful factors from different backends remain in a comparable signal range. This supports the view

Table 4: Repair-aware code-agent backend robustness on the same 100 schema plans. Kimi is reported under pass@1 only because retry was not available in this export.

Backend	Pass@1	Repair	Final	$\lvert\textbf{RankIC}\rvert \uparrow$	RankIC
GPT-5.4	97/100	2	99/100	0.0116	0.0040
GPT-5.4-mini	81/100	10	91/100	0.0129	0.0073
Kimi-K2.7-Code	82/100	0	82/100	0.0138	0.0075
DeepSeek-V4-Flash	63/100	30	93/100	0.0168	0.0121
Qwen3.6-Flash	49/100	40	89/100	0.0161	0.0115
GLM-5.2	75/100	22	97/100	0.0145	0.0109

Table 5: Full predictability results for schema-space reward prediction. Top10% lift is measured against random selection.

Feature Set	Model	ρ_S	ρ_P	RMSE↓	Top10% Lift
Mean baseline	Mean	0.000	0.000	0.0908	1.000
Single schema	Ridge	0.212	0.214	0.0892	1.413
Single schema	ElasticNet	0.214	0.217	0.0888	1.381
Single schema	LGBM	0.237	0.245	0.0884	1.506
Single schema	MLP	0.121	0.123	0.0963	1.295
Schema + pair interaction	LGBM	0.239	0.250	0.0884	1.519
Schema + pair interaction	MLP	0.204	0.201	0.0903	1.273
Shuffled reward	LGBM	0.029	0.021	0.0917	0.946

that the schema provides transferable implementation intent rather than relying on a single agent backend.

Predictability of Schema Space

The predictability experiment evaluates whether reward can be inferred from the discrete schema representation alone. The dataset contains 12,126 valid plan–reward pairs and 11,295 unique schema keys. We use an 80/20 split, resulting in 2,430 held-out test examples. The target reward follows the historical factor-mining reward used in the search logs:

$$r = \frac{10\mathrm{IC} + \mathrm{ICIR} + 10\mathrm{RankIC} + \mathrm{RankICIR}}{4}. \quad (14)$$

The feature construction intentionally excludes schema text embeddings, TF–IDF features, generated code, factor values, and backtest trajectories. The two main feature sets are: (i) single-schema indicators with category/scope counts; and (ii) schema-pair features with pairwise schema and category-family-scope interactions. The mean baseline predicts a constant reward, while the shuffled-reward control preserves the feature matrix and model class but randomly permutes reward labels.

The shuffled-reward control remains near random, supporting that the observed predictability reflects a genuine schema–reward relationship rather than feature dimensionality alone.

The corresponding visualization is reported in Figure 5 in the main paper. It shows the same control logic: schema features yield nontrivial ranking and top-decile enrichment, pair interactions add a small gain, and shuffled rewards collapse toward random selection.